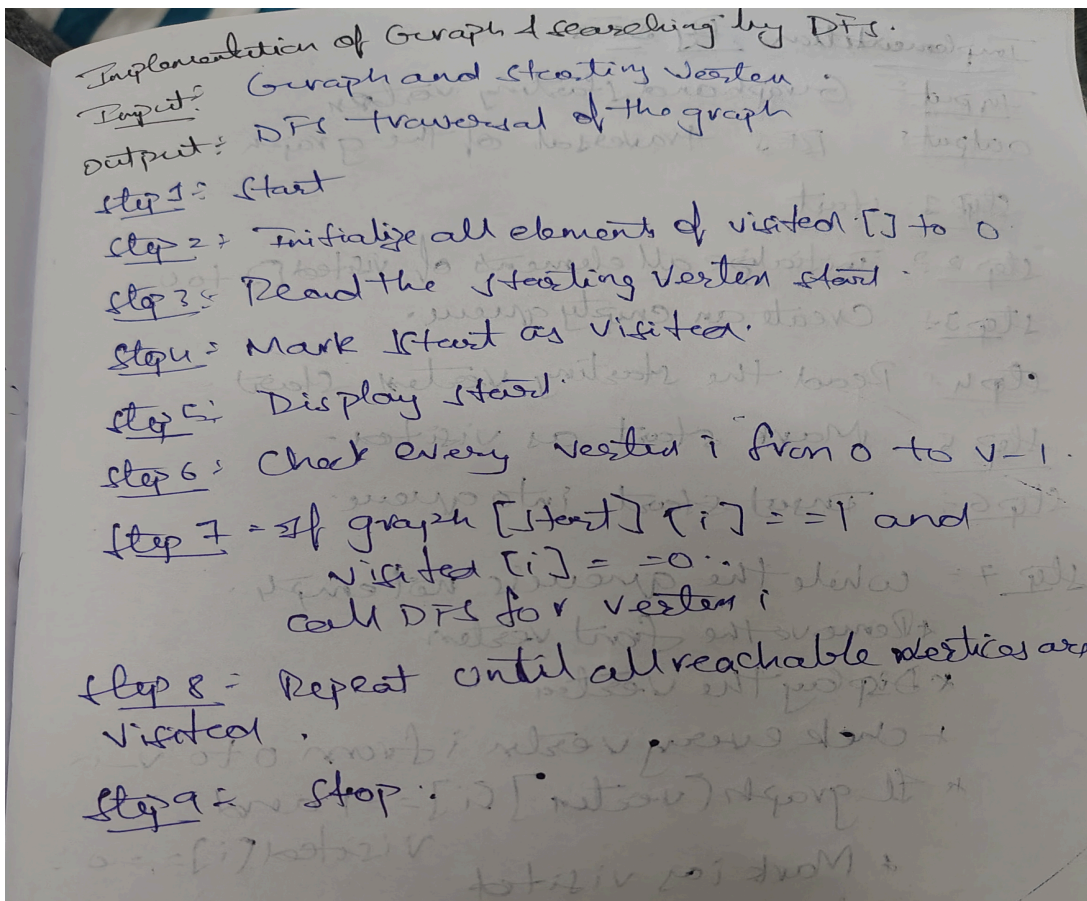


## PRACTICAL-8

**AIM:** Implementation of Graph and Searching (DFS and BFS).

### Depth first search

#### ALGORITHM:



**CODE:**

```
#include <iostream>
using namespace std;

int graph[100][100];
int visited[100];
int V;

void DFS(int v)
{
    visited[v] = 1;
    cout << v << " ";

    for (int i = 0; i < V; i++)
    {
        if (graph[v][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}

int main()
{
    int E, u, v, start;

    cout << "Enter number of vertices: ";
    cin >> V;

    // Initialize graph
    for (int i = 0; i < V; i++)
    {
        for (int j = 0; j < V; j++)
        {
            graph[i][j] = 0;
        }
    }

    cout << "Enter number of edges: ";
    cin >> E;
```

Department of Computer Engineering  
01AI0506 Design and Analysis of Algorithm Lab Manual

```
cout << "Enter edges (u v):" << endl;

for (int i = 0; i < E; i++)
{
    cin >> u >> v;

    graph[u][v] = 1;
    graph[v][u] = 1;
}

cout << "Enter starting vertex: ";
cin >> start;

// Initialize visited
for (int i = 0; i < V; i++)
{
    visited[i] = 0;
}

cout << "DFS Traversal: ";
DFS(start);

return 0;
}
```

### OUTPUT:

```
Enter number of vertices: 6
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 5
Enter starting vertex: 0
DFS Traversal: 0 1 3 4 2 5
```

## TIME COMPLEXITY:

Time Complexity

```

void DFS (int v)
{
    visited [v] = 1;
    cout << v;
    for (int i = 0; i < v; i++)
    {
        if (graph[v][i] == 1 && !visited[i])
            DFS (i);
    }
}
    
```

$\sim$  vertices &  $\sim$  no. of times it checks

$T(n) = n \times n$

$\rightarrow T(n) = O(n^2)$  — Best Case

Average Case — DFS still visits vertices and scans their adjacency matrix rows.

$n \times n = n^2$

Average =  $O(n^2)$

Worst Case — If the graph is highly connected, DFS can visit all  $n$  vertices and scan all  $n$  partitions for each.

$n \times n^2$

Worst Case =  $O(n^3)$

## Method 2: Minimum number of ways

## Breadth first search

### ALGORITHM:

Implementation By BFS

Input: Graph and starting vertex

Output: BFS traversal of the graph

Step 1: start

Step 2: Initialize all elements of visited[] to 0.

Step 3: Create an empty queue.

Step 4: Read the starting vertex start.

Step 5: Mark start as visited.

Step 6: Insert start into queue.

Step 7: while the queue is not empty.

- \* Remove the front vertex
- \* Display the vertex
- \* check every vertex  $i$  from 0 to  $v-1$
- \* If  $\text{graph}[\text{vertex}][i] = 1$  and  $\text{visited}[i] = 0$ .
- \* Mark  $i$  as visited
- \* Insert  $i$  into the queue

Step 8: Repeat until queue becomes empty

Step 9: Stop

## CODE:

```
#include <iostream>
#include <queue>
using namespace std;

int graph[100][100];
int visited[100];
int V;

void BFS(int start)
{
    queue<int> q;

    visited[start] = 1;
    q.push(start);

    while (!q.empty())
    {
        int v = q.front();
        q.pop();

        cout << v << " ";

        for (int i = 0; i < V; i++)
        {
            if (graph[v][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                q.push(i);
            }
        }
    }
}

int main()
{
    int E, u, v, start;

    cout << "Enter number of vertices: ";
```

```
cin >> V;

// Initialize graph
for (int i = 0; i < V; i++)
{
    for (int j = 0; j < V; j++)
    {
        graph[i][j] = 0;
    }
}

cout << "Enter number of edges: ";
cin >> E;

cout << "Enter edges (u v):" << endl;

for (int i = 0; i < E; i++)
{
    cin >> u >> v;

    graph[u][v] = 1;
    graph[v][u] = 1;
}

cout << "Enter starting vertex: ";
cin >> start;

// Initialize visited
for (int i = 0; i < V; i++)
{
    visited[i] = 0;
}

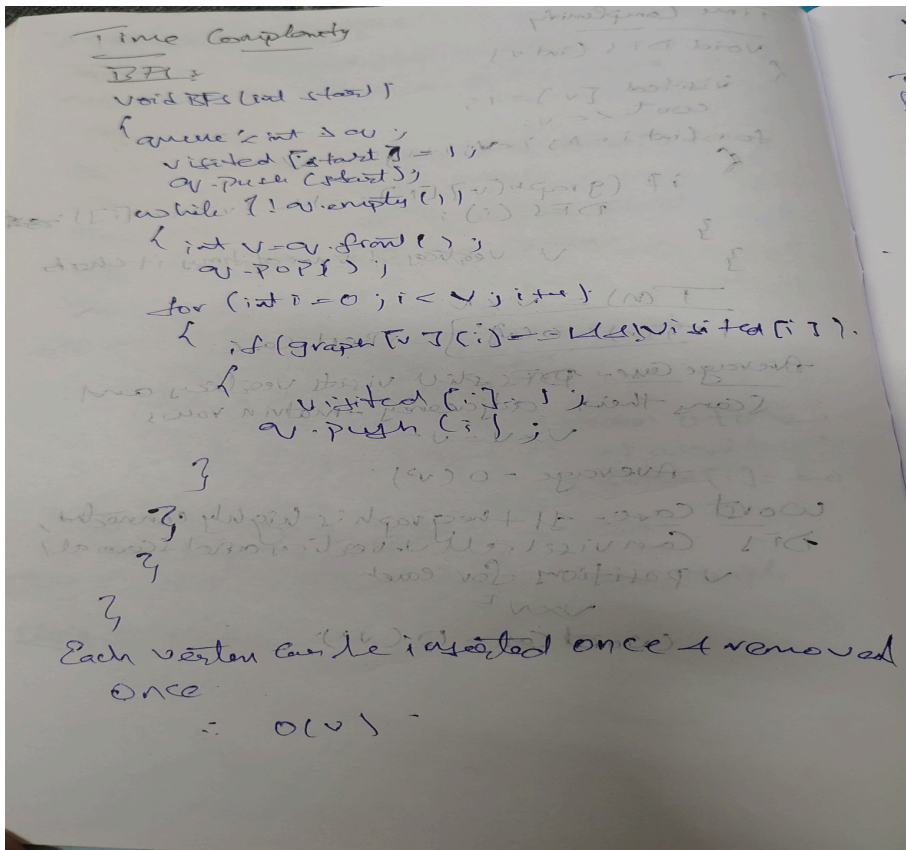
cout << "BFS Traversal: ";
BFS(start);

return 0;
}
```

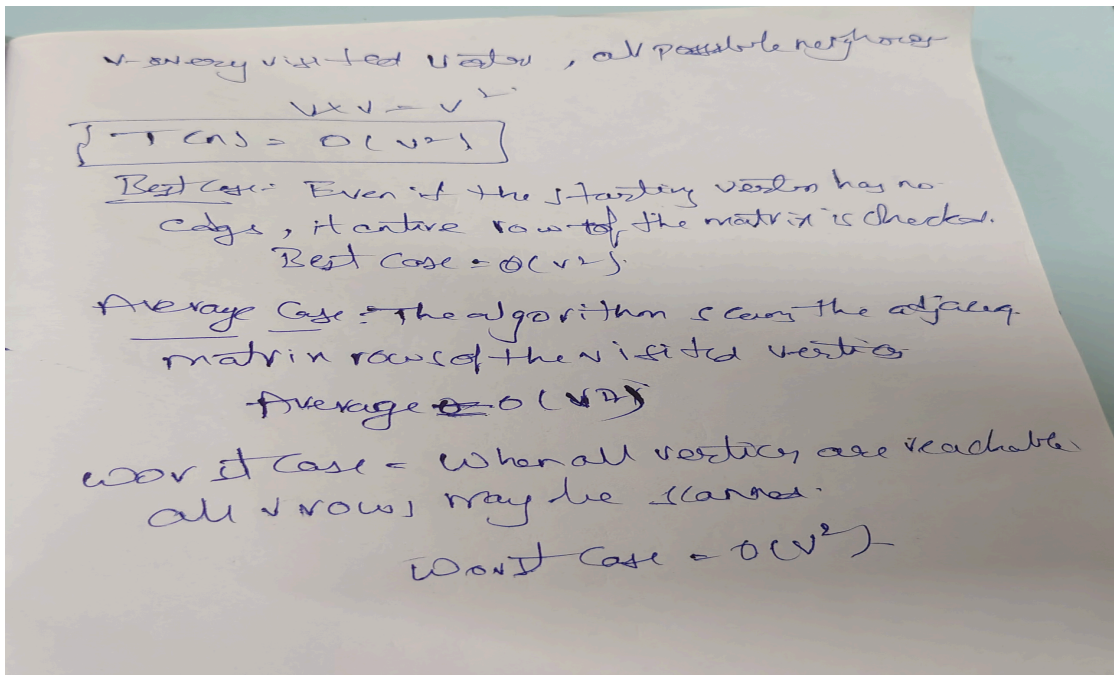
## OUTPUT:

```
Enter number of vertices: 6
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
1 4
2 5
Enter starting vertex: 0
BFS Traversal: 0 1 2 3 4 5
```

## TIME COMPLEXITY:



Department of Computer Engineering  
01AI0506 Design and Analysis of Algorithm Lab Manual



## CONCLUSION:

The Graph Searching practical was successfully implemented using DFS (Depth First Search) and BFS (Breadth First Search). DFS explores the graph deeply using recursion, while BFS explores the graph level by level using a queue. Using an adjacency matrix, both algorithms have a time complexity of  $O(V^2)$ .